SISTEMA DE LISTA DE TAREFAS –

Documentação

Este projeto consiste em um sistema de lista de tarefas, desenvolvido com HTML, CSS e JavaScript.

O sistema permite cadastrar, visualizar, editar, excluir e concluir tarefas, além de armazenar os dados no navegador utilizando o *LocalStorage*.

1. Estrutura HTML:

O HTML foi organizado utilizando tags semânticas como:

- <header> → cabeçalho com título e campo de busca
- <main> → conteúdo principal
- <section> → separação entre formulário e tarefas
- <form> → cadastro de tarefas

1.1 Formulário

O seguinte formulário é controlado pelo JavaScript:

```
<form id="taskForm">
```

O “id="taskForm" permite que o jvascript capture os dados digitados pelo usuário.

1.2 Validação com “required”

```
<input type="text" id="title"
placeholder="Título da tarefa" required>
```

O “required” impede que o usuário envie o formulário vazio — isso atende o requisito de validação do trabalho.

2. Estilização CSS

2.1 Reset

```
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family: Arial, Helvetica, sans-serif;
}
```

Remove estilos padrão do navegador, garantindo um layout mais consistente.

2.2 Uso do flexbox

```
body{
background:#f4f6fb;
color:#333;
display:flex;
flex-direction:column;
min-height:100%;
}
```

O Flexbox foi utilizado para:

- Organizar os elementos em coluna
- Criar espaçamentos (`gap`)
- Alinhar elementos

2.3 Prioridade com cores

```
.baixa{
border-left:6px solid #4CAF50;
}
.media{
border-left:6px solid #ff9800;
}
```

```
.alta{
border-left:6px solid #f44336;
}
```

Cada tarefa recebe uma classe diferente conforme a prioridade, facilitando a visualização.

2.4 Classe “.done”

```
.done{
text-decoration: line-through;
opacity: 0.6;
background-color: #d4edda;
}
```

Essa classe é aplicada quando a tarefa é concluída, deixando ela “riscada”.

3. JavaScript

3.1 Armazenamento com LocalStorage

Através do “LocalStorage”, os dados são salvos no navegador;

O “JSON.parse()” transforma o texto salvo em objeto novamente.

```
const savedTasks = localStorage.getItem("tasks");

if (savedTasks) {
    tasks = JSON.parse(savedTasks);
}
```

Isso permite que as tarefas não sejam perdidas ao atualizar a página.

3.2 Salvando dados

Com o “JSON.stringify()”, o array é transformado em texto, que é salvo no navegador.

```
function saveTasks() {
    localStorage.setItem("tasks",
JSON.stringify(tasks));
}
```

3.3 Evento de envio do formulário

```
form.addEventListener("submit", function (event) {

    event.preventDefault();
```

O “preventDefault()” impede o recarregamento da página e permite controlar tudo via javascript.

3.4 Criação da tarefa (objeto)

```
    const task = {
        title: title,
        description: description,
        date: date,
        priority: priority,
        status: "Pendente"
    };
```

Cada tarefa é armazenada como um “objeto”, facilitando a manipulação dos dados.

3.5 Atualização dinâmica da tela

```
function displayTasks(taskArray = tasks) {
```

```
    taskList.innerHTML = "";
```

Com essa função, ela:

- Limpa a tela antes de redesenhar;
- Evita duplicação de tarefas.

3.6 Criação de elementos (DOM)

Aqui o javascript cria os elementos HTML dinamicamente:

```
        const taskElement =
document.createElement("div");
```

3.7 Inserindo conteúdo com template string

O *template string* permite inserir dados da tarefa diretamente no HTML.

```
        taskElement.innerHTML = `
<h3>${task.title}</h3>
```

3.8 Botões com eventos

3.8.1 Concluir tarefa

Altera o status e atualiza a interface:

```
            task.status = "Concluída";
```

3.8.2 Editar/ Excluir tarefa

Essa linha remove a tarefa antiga e preenche o formulário com os dados para edição:

```
            tasks.splice(index, 1);
```

3.8.3 Busca de tarefas

Este trecho filtra tarefas pelo título e usa “toLowerCase()” para evitar problemas com letras maiúsculas/minúsculas:

```
    const filteredTasks = tasks.filter(function
(task) {
        return
task.title.toLowerCase().includes(termo);
    });
```